Most MaxSim Winners Flip, Retrieval Survives: Low-Margin Substitution in Sign-Coded Late Interaction

Anirudh Lakhotia[1], Nischal Helagally Shantharaju[1]

[1]Couchbase, Inc., USA

Abstract

Late-interaction retrieval appears unusually robust to extreme compression. We study a sub-bit sign-coded ColBERT representation that stores each 128-dimensional document token as the 64 signs of a projected vector (0.5 bits per original dimension, 8 B/tok). On a 100k-passage MS MARCO diagnostic slice, sign coding changes the winning token for roughly 70% of query tokens in a pooled top-100 diagnostic, yet relevant-document recall remains near the fp32 ceiling and MRR@10 is recoverable by a short fp32 rerank. We investigate this robustness through *low-margin substitution*: although sign coding frequently changes the winning document token, the replacement usually has a similar fp32 score, so the resulting loss in the MaxSim sum remains small. The remaining retrieval loss largely reflects changes in the ordering of near-tied, top-ranked candidates rather than relevant documents dropping out of the candidate set. The mechanism also suggests that projection learning has limited room to help once errors are dominated by low-margin substitutions. Consistent with this prediction, on the full 8.8M-passage MS MARCO corpus a trained projection does not improve MRR@10 over a random orthogonal projection at matched storage, with nearly identical Recall@1000. These findings suggest that extreme compression for late-interaction retrieval need not faithfully preserve token-level geometry. A simple projection-training-free recipe – a random 64-dimensional projection, 8 B/tok sign-coded document storage, and fp32 reranking of the top 100 candidates – recovers fp32 MRR@10 on the 100k slice. The sign code is a compact per-token storage format, complementary to pruning-based retrieval engines. Code and artifacts are available at https://github.com/anirudhlakhotia/subbit-multivector-retrieval.

Keywords

ColBERT, late interaction, MaxSim, sign quantisation, low-margin substitution, ColBERT compression, reproducibility

1. Introduction

Late-interaction retrievers such as ColBERT [1, 2] keep one embedding per document token and score a query–document pair with MaxSim, $s(Q, D) = \sum_i \max_j q_i \cdot d_j$: each query token is compared against every document token, only the largest similarity is kept, and these per-query-token maxima are summed into the final score. Because each document token carries a 128-dimensional fp32 embedding, the index is large: on the full 8.8M MS MARCO passage corpus it runs to several hundred gigabytes. This storage cost has motivated increasingly aggressive per-token compression, and several production multi-vector systems already offer sign-coded document tokens [3, 4, 5], keeping only the sign of each coordinate and discarding magnitude. We study a variant that goes further, first projecting each document token to r=64 dimensions and keeping only the signs of the projection: 64 sign bits for the 128 original dimensions, or 0.5 bits per dimension (hence sub-bit), 8 bytes per token. This perturbs token-level similarities heavily, yet retrieval effectiveness stays close to the fp32 reference.

On a 100k-passage MS MARCO slice that we use for token-level diagnostics (end-to-end retrieval is also evaluated on the full 8.8M corpus), this r=64 code changes the document token that achieves the maximum similarity for roughly 70% of query tokens. We measure these winner changes over the pooled tokens of each query's top-100 fp32 candidate passages, the head of the ranking that MRR@10 and Recall@100 depend on.

Many compression methods are designed to preserve or estimate similarity: ColBERTv2 stores centroid ids with quantised residuals [2], and PLAID serves that representation with centroid interaction

ReNeuIR at SIGIR 2026: The Fifth Workshop on Reaching Efficiency in Neural Information Retrieval, July 24, 2026, Melbourne, Australia

anirudh.lakhotia@couchbase.com (A. Lakhotia); nischal.hs@couchbase.com (N. H. Shantharaju)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

and pruning [6]; EMVB combines bit-vector filtering with product quantisation [7]; ITQ learns an orthogonal rotation before thresholding [8]; and RaBitQ uses a random rotation with correction terms to estimate distances or inner products with error bounds [9]. The sign code studied here is different: it discards magnitude and uses no reconstruction or correction term during candidate scoring, so individual token similarities can drift substantially from their fp32 values. That retrieval survives anyway suggests that robustness does not require faithful per-token similarities, pointing instead to a property of the MaxSim aggregation itself.

To understand the discrepancy, we examine the document tokens that replace the original MaxSim winners. They are rarely arbitrary: the new winner is usually an alternative whose fp32 score was already close to the original's, so the swap changes that query token's contribution only slightly, and even widespread winner changes move the summed score little. Winners with a clear lead, where a swap would be costly, are rare. We call this *low-margin substitution*; the $r{=}64$ sub-bit regime, which discards both magnitude and half the dimensions, is our primary test of this mechanism.

If low-margin substitution explains this robustness, exact fp32 geometry is not necessary; it may be enough for replacement tokens to remain close to the original winners in fp32 score. Under this view, learning has limited room to help once random sign coding already keeps most substitutions low-margin. Consistent with this prediction, at full 8.8M scale training does not improve MRR@10 over a random orthogonal projection (§5). MRR@10 is the mean reciprocal rank of the first relevant passage in the top ten.

The practical consequence is a recipe that needs no projection training. A random orthogonal projection to $r{=}64$ generates candidates, and a full-dimensional rerank of the top 100 recovers fp32 MRR@10 at 100k and reaches the published ColBERTv2 reference level at 8.8M, with no centroid table (§9). On the 100k diagnostic corpus the sign code reaches fp32-level MRR@10, after the rerank, from a per-token candidate index 2.25–$8.25\times$ smaller than ColBERTv2's residual codec; as a per-token storage format it is complementary to pruning-based engines like PLAID, not a replacement for them (§9.1).

Contributions. We study sign-coded ColBERT in a sub-bit regime ($r{=}64$, 0.5 bits per original dimension), where each document token is represented only by the signs of a low-rank projection, and we ask what survives, why, and whether the projection must be learned.

- **The phenomenon.** On the 100k MS MARCO slice, sign coding changes the MaxSim winner for a majority of query tokens, yet relevant-document recall stays near the fp32 reference and MRR@10 is recovered by reranking; across eight out-of-domain (for the encoder) BEIR collections with fp32 baselines, sign coding retains most of fp32's NDCG@10 (§5, §6).
- **The mechanism.** Most winner changes are low-margin substitutions between near-tied document tokens, so each query token's contribution barely moves. We characterise how these substitutions add up: pooled-token diagnostic sums stay stable and relevant documents stay in the candidate set despite frequent winner changes (§7).
- **No detectable learning gain at full scale.** At full 8.8M MS MARCO scale a learned low-rank projection does not improve MRR@10 over a random orthogonal one and gives nearly identical Recall@1000 (§5).
- **The recipe.** A random orthogonal projection with a full-dimensional rerank of the top candidates requires no projection training. On the 100k diagnostic corpus it recovers fp32-level MRR@10; at 8.8M it reaches the published ColBERTv2 reference level (§9). Its 8 B/tok candidate index is 2.25–$8.25\times$ smaller per token than the evaluated ColBERTv2 residual-codec configurations. This per-token storage format complements pruning-based engines such as PLAID (§9.1).

2. Related work

Work on late-interaction retrieval cuts cost in three ways: storing fewer vectors, compressing each vector, or serving compressed representations more efficiently. Each asks whether retrieval quality

survives compression. Here we ask which token-level MaxSim properties must survive aggressive compression, particularly when the winning document token changes.

Efficient late-interaction retrieval. Several systems compress and serve the centroid index. ColBERTv2 compresses token embeddings into centroid ids with quantised residuals [2]; PLAID accelerates search over that representation with centroid interaction and pruning [6]. EMVB adds bit-vector prefiltering and uses product quantisation in place of PLAID's residual compressor [7]. WARP serves XTR-trained retrievers through implicit decompression that avoids vector reconstruction [10]. TACHIOM scales token-aware centroid allocation to millions of centroids and scores documents from those centroids directly [11]. This line improves how token representations are indexed and searched. It validates end-to-end quality at each approximation stage, but does not isolate which token-level interactions a ranking depends on.

Vector-count reduction. A separate line of work reduces the number of document vectors. Some methods prune tokens: a study of token-dropping strategies showed that ColBERT passage indexes can be pruned by up to 30% at small cost [12], Zong and Piwowarski regularise for score preservation while keeping about 30% of document tokens [13], and static low-IDF token removal [14] and a Voronoi-cell influence criterion [15] thin the index. Others produce fewer vectors by construction: ColBERTer learns whole-word representations and drops vectors that do not contribute to scoring [16], ConstBERT fixes a constant number of vectors per document [17], query-agnostic fixed-budget compression [18] and CRISP's clustering [19] also reduce it, token pooling merges similar document vectors [20], and MUVERA collapses the multi-vector set into a fixed-dimensional encoding with random space partitions for candidate generation, reranking candidates with exact MaxSim [21]. Where these methods change how many vectors are scored, we keep the token structure intact and compress each token's representation.

Token compression and learned projections. Two families compress the individual vector. Learned codes minimise reconstruction or ranking error: trained codebooks in PQ, learned rotations in ITQ and OPQ, and supervised codes in BPR and JPQ [22, 8, 23, 24, 25]. A data-oblivious family instead draws random projections independent of the data. SimHash [26] and the broader random-projection and hashing literature [27, 28, 29] need no training data, and RaBitQ [9] pairs a random orthogonal rotation with binary data codes and stored per-vector correction terms, yielding an unbiased distance/inner-product estimator with high-probability per-pair error bounds rather than a rank-order guarantee. CoRECT shows that non-learned multi-bit scalar quantisation scales to 100M passages in single-vector retrieval with no statistically significant quality loss [30]; whether this carries to token-level MaxSim is open. Closer to our setting, Clavié et al. train ColBERT variants with alternative projection blocks and evaluate them under PLAID indexing with 4-bit residuals, reporting NDCG@10 gains on TREC-DL19 and three of four out-of-domain BEIR subsets [31], comparing variants within a learned-projection family. We instead study sign codes derived from low-rank projections, including fully random ones. Our goal is not to improve similarity preservation but to understand which properties of MaxSim must be preserved when individual similarities are heavily perturbed. To our knowledge, no prior work directly compares trained and random projections for sign-coded late-interaction retrieval across both the sub-bit low-dimensional setting ($r{=}64$) and the full-dimensional one-bit setting ($r{=}128$). ITQ performed the analogous comparison for single-vector binary codes [8]; we perform it for late interaction at corpus scale (§5).

Reranking and compressed serving. One-bit token representations for late interaction are already well established in both research and production: ColBERTv2's $b{=}1$ mode [2] and Bi-ColBERT [32] on the research side, and deployed systems on the production side. Vespa and Elasticsearch sign-code per-token ColBERT and ColPali document vectors to 1 bit per dimension, $32\times$ smaller than fp32, scoring MaxSim in Hamming space or asymmetrically against full-precision queries [3, 33, 4]. Qdrant

reports faster search than scalar quantisation at similar accuracy in a ColPali experiment using binary quantisation with oversampling and rescoring [5].

These systems mostly use binary or one-bit document-token representations; we study a more aggressive regime that first projects each document token to a lower-dimensional space and then sign-codes it. Rescoring is itself a standard two-stage pattern: BPR retrieves candidates by Hamming distance over binary passage codes and rescores them with the continuous query embedding [22], and cascade rankers rerank a cheap first-stage candidate set with a stronger model [34, 35]. The closest comparison point is SPLATE, which matches PLAID quality on ColBERTv2 by rescoring the top 50 candidates from a SPLADE-style sparse adapter [36]. Our pipeline follows the same shape, using low-rank sign-coded MaxSim for candidate generation and fp32 MaxSim for reranking. These systems show that aggressive compression can preserve a useful candidate set when followed by a higher-precision reranker. Why that candidate set survives is the question we take up.

Understanding MaxSim. MaxSim decomposes into one argmax per query token, and prior analyses have studied how those matches are formed and which tokens carry the score. XTR, the retriever WARP serves, retrains the encoder so that token retrieval surfaces the document tokens that matter for scoring, letting documents be ranked from retrieved tokens alone [37], and a matching analysis traces the score to co-occurring, high-importance token pairs [38]. Weighted Chamfer scoring adds IDF-based or few-shot-trained token weights to the sum over query tokens [39], and an empirical analysis characterises MaxSim's length bias and the negligible signal beyond the top-1 document token [40]. CrossQ also examines winner flips under learned quantisers, observes that replacing a winner need not substantially change its score, and allocates precision to likely winners to reduce flips [41]. We instead ask why retrieval remains effective despite widespread winner changes under sign coding. We argue that, in our compressed setting, this robustness is consistent with low-margin substitution.

3. Method

3.1. Representation

Let $d \in \mathbb{R}^{128}$ be a ColBERTv2 document token. The sign-coded encoding is

$$\mathrm{enc}_D(d) \;=\; \mathrm{sign}(R\,d) \;\in\; \{-1,+1\}^r, \tag{1}$$

where $R \in \mathbb{R}^{r\times 128}$ is a fixed projection; the resulting sign code uses $r/8$ bytes per token. We compare three choices of R at the same byte budget per token: *identity truncation* ($R = [I_r \mid 0]$), *random orthogonal* (a fixed row-orthonormal matrix, $RR^\top = I_r$, drawn once at a given seed), and *trained* (learned using the objective in §3.3). The three variants share this byte budget and differ only in R (each optionally carrying the same query-side scale head, §3.3, which adds no per-token storage). Most of the analysis follows r=64.

3.2. Retrieval scoring

Documents are stored as sign codes, but queries stay in fp32 (asymmetric scoring): $\mathrm{enc}_Q(q) = R\,q \in \mathbb{R}^r$ is a float query projection dotted against the binary document codes. The score is the standard MaxSim, which matches each query token to its single best document token and sums those matches,

$$s(Q,D) \;=\; \sum_{i=1}^{m} \max_{j=1,\dots,n} \mathrm{enc}_Q(q_i)\cdot \mathrm{enc}_D(d_j), \tag{2}$$

computed on the stored sign codes. Scoring optionally includes a small learned query-side scale head: a scalar gain $S(q) \in [1, 1.5]$ computed from the 128-dim query token (129 parameters; §3.3) that multiplies the projected query token. It is query-side, so it adds no per-token storage and can

accompany any choice of R; the main 8.8M comparison (Table 1) gives all three the same scale-head architecture and training opportunity, isolating R. We use this asymmetric scoring throughout.

The projection R is shared across all tokens and stored once for the entire index.[1]

We take $r{=}64$ (0.5 bits/dim, 8 B/tok) as the primary operating point and also report $r{=}128$ (1 bit/dim, 16 B/tok).

At evaluation the ColBERTv2 encoder is frozen and R is held fixed (at the seed, the identity, or the trained checkpoint). Scoring is therefore ordinary MaxSim: the sign code changes only how document tokens are represented, not how their scores are aggregated.

3.3. Training objective

The trained variant learns a projection R that approximates selected fp32 MaxSim scores of the original model under sign coding, training on (query, positive, negative) triples with the ColBERTv2 encoder frozen. The score a query assigns a document is determined by the few document tokens that compete to be each query token's best match. The objective therefore has three components: a top-K anchor that preserves those competitors, a guard that discourages sign coding from introducing spurious new winners, and an orthogonality regulariser that keeps the projection well-conditioned.

Formally, for a query token q_i and document token d_j, let $s^T_{ij} = q_i \cdot d_j$ denote the teacher (fp32) score and $s^S_{ij} = r^{-1/2}\,\mathrm{enc}_Q(q_i) \cdot \mathrm{enc}_D(d_j)$ the sign-coded student score (the $r^{-1/2}$ factor removes the growth of the sign-vector norm with r and gives the squared-error terms below a comparable scale; as a global constant it leaves ranking unchanged, so the scorer in Eq. 2 omits it). Let $\mathcal{T}^K_i = \mathrm{argtop}^K_j\, s^T_{ij}$ be the teacher's K highest-scoring document tokens for q_i.

The *top-K anchor* matches the student's scores to the teacher's on $\mathcal{T}^K_i$, the document tokens most likely to win each query token's MaxSim. Because MaxSim depends only on the best-matching document token, these competitive tokens are a natural place to focus the distillation:

$$\mathcal{L}_{\text{topk}} \;=\; \frac{1}{m}\sum_{i=1}^{m}\sum_{j\in\mathcal{T}^K_i}\left(s^S_{ij} - s^T_{ij}\right)^2; \tag{3}$$

we use $K{=}5$.

The *guard* discourages a previously uncompetitive token from becoming the student's winner solely because sign coding perturbs the scores: it anchors the strongest such outsider back to its teacher score,

$$\mathcal{L}_{\text{guard}} \;=\; \frac{1}{m}\sum_{i=1}^{m}\left(s^S_{ij^*_i} - s^T_{ij^*_i}\right)^2, \quad j^*_i = \arg\max_{j\notin\mathcal{T}^K_i} s^S_{ij} \text{ (stop-grad)}. \tag{4}$$

The *orthogonality regulariser* discourages R from collapsing several coded dimensions onto the same direction: $\mathcal{L}_{\text{ortho}} = \|RR^\top - I_r\|^2_F$.

Each term is averaged over the positive and negative document of a training triple, written $\mathcal{L}^{\pm}$. The composite objective is

$$\mathcal{L} \;=\; \lambda_{\text{topk}}\,\mathcal{L}^{\pm}_{\text{topk}} + \lambda_{\text{guard}}\,\mathcal{L}^{\pm}_{\text{guard}} + \lambda_{\text{ortho}}\,\mathcal{L}_{\text{ortho}}, \tag{5}$$

with $\lambda_{\text{topk}} = \lambda_{\text{guard}} = 1$ and $\lambda_{\text{ortho}} = 0.001$. The scale head produces a single multiplier in $[1, 1.5]$, a function of the 128-dim query token, that rescales the projected query token (129 parameters). Every trained-R result uses this scale head unless labelled "no scale". A random R carries its own trained head in all 8.8M runs and in the per-query trained-vs-random comparisons of §8; the random-R mechanism analyses (§6–§7) and the 100k rerank recipe (Table 7) use none.

[1] R costs $r \times 128 \times 4$ bytes once (32 KiB at $r{=}64$), under 5×10^{-3} B/tok beyond ∼100k passages.

4. Experimental setup

On MS MARCO we compare trained, random, and identity projections at matched storage budgets. On BEIR we use identity vs. random as a non-learned projection-choice comparison, and fp32 to compare retention.

Query convention. We use standard ColBERTv2 augmented queries throughout: the 100k diagnostic slice, the 8.8M corpus, and BEIR all follow this convention.

Scale	Use
50k triples	training budget
100k passages	diagnostic analysis
8.8M passages	full-corpus evaluation

Training. R and the 129-parameter scale head are trained with AdamW (learning rate 10^{-3}, weight decay 10^{-4}), batch size 32, gradient clipping at 1.0, and a cosine schedule with 250 warmup steps over 10 epochs (15,620 steps), keeping the step-15,000 checkpoint, which had the best MRR@10 on the 100k slice's dev queries, the same queries we evaluate on. R is trained on 50,000 (query, positive, negative) triples — 10,000 query–positive pairs from the MS MARCO passage-train qrels restricted to the 100k-passage training subset, each paired with five non-qrel negatives drawn uniformly at random from the same subset (seed 42), with no mined hard negatives. All reported trained-R results use seed 42. In the 8.8M comparison the random baseline uses a separately trained scale head with R frozen to a random orthonormal matrix. Training and the 100k diagnostic evaluation run on an Apple M4 Pro (48 GB); the full 8.8M retrieval evaluation runs on a single NVIDIA A100-80GB.

Evaluation. We evaluate on MS MARCO Passage [42] dev queries (6,980) against the full 8.8M corpus and the 100k diagnostic slice, and on nine BEIR domains [43]: nfcorpus, scifact, arguana, scidocs, fiqa, trec-covid, webis-touche2020, quora,[2] and nq. The four larger Wikipedia-derived corpora (hotpotqa, dbpedia-entity, fever, climate-fever) were excluded because they exceeded our encoding budget. The 100k slice contains all 7,433 passages judged relevant for the dev queries plus the first 92,567 distinct train-qrel passages in dataset order. The encoder is the public `colbert-ir/colbertv2.0` checkpoint. All first-stage retrieval is exact. Asymmetric MaxSim between float queries and sign-coded documents is computed exhaustively over all 8,841,823 passages, with no ANN index or candidate pruning. The PLAID rows in Table 1 isolate PLAID's centroid+b-bit residual *codec* (C=32,768): we apply it to the shared frozen embeddings and score it with the same exhaustive MaxSim harness as every other row, dropping PLAID's candidate generation and centroid pruning so that the compression scheme is the only factor varying across rows. Run as a full engine over the same passages (independently re-encoded), official PLAID agrees with our codec within 0.004 MRR@10 (0.8593 codec vs. 0.8631 engine, Table 8). The ITQ and RaBitQ rows are likewise our own implementations on this dev slice; the PQ and OPQ rows use Faiss [44]. Two-stage rows (§9) rerank the exact top-K candidates ($K \in \{100, 1000\}$) with full-dimensional MaxSim (fp32 at 100k, fp16 at 8.8M). We report r=64 (0.5 bits/dim) as the primary operating point and r=128 (1 bit/dim) alongside it. We report MRR@10 and Recall@$\{100, 1000\}$ on MS MARCO and NDCG@10 on BEIR; statistical uncertainty on MS MARCO is a per-query paired bootstrap over the 6,980 dev queries (10,000 resamples). Reported storage counts include only the per-token code and exclude amortised projection matrices and centroid codebooks.

5. Does learning the projection matter?

The central question left open by Section 3 is whether learning the projection improves retrieval quality. Table 1 shows that on the full 8.8M MS MARCO corpus it does not improve MRR@10: trained and

[2] Quora is evaluated on its dev split (5,000 queries). The other eight corpora use their test splits.

random orthogonal projections give nearly the same MRR@10 at matched storage. We then examine a smaller diagnostic slice and a cross-domain BEIR panel to understand the scope of this result.

Two distinct comparisons recur: trained vs. random on the 100k slice and at 8.8M, and, on BEIR, identity vs. random as a non-learned projection-choice comparison.

Table 1
Storage–quality across compression methods on MS MARCO dev. **Top:** 100k diagnostic slice. **Bottom:** full 8.8M corpus, asymmetric scoring. The matched-byte comparison is trained R vs. random orthogonal R at r=64, 8 B/tok, where at 8.8M their MRR@10 differs by only 0.0001. PLAID, ITQ, and RaBitQ are our own implementations, and PQ and OPQ use Faiss (§4); storage is per-token (†RaBitQ 24=16+4+4 B; ‡published 8.8M ColBERTv2 result, a reference from a different harness).

Method	B/tok	MRR@10	Rec@100	Rec@1000
100k MS MARCO dev				
fp32 ColBERTv2	512	0.8641	0.994	0.9989
PLAID b=4 [6]	66	0.8627	0.994	0.9989
PLAID b=2	34	0.8593	0.993	0.9988
PLAID b=1	18	0.8498	0.993	0.9987
ITQ r=64, asym [8]	8	0.8319	0.9871	0.9958
PQ 8×8 [23], ADC	8	0.8330	0.9867	0.9954
OPQ 8×8 [24]	8	0.8048	0.9802	0.9947
RaBitQ [9], asym	24†	0.8582	0.9925	0.9982
Trained R, r=64 + scale, asym	8	0.8496	0.991	0.9972
Random orthogonal R, r=64 asym	**8**	**0.8393**	0.990	0.9963
Full 8.8M MS MARCO dev, asymmetric scoring				
RaBitQ	24†	0.390	0.909	0.982
ITQ r=64	8	0.369	0.889	0.973
main comparison: trained vs. random vs. identity, r=64 + scale (8 B/tok)				
Trained R, r=64 + scale	**8**	**0.3723**	0.900	0.975
Random orthogonal R, r=64 + scale	**8**	**0.3724**	0.892	0.973
Identity R, r=64 + scale	**8**	**0.3735**	0.894	0.975
control: r=128 + scale (16 B/tok), only R changes				
Trained R, r=128 + scale	16	0.3889	0.912	0.982
Random orthogonal R, r=128 + scale	16	0.3916	0.911	0.983
Identity R, r=128 + scale	16	0.3921	0.910	0.983
reference: different evaluation harness				
Published ColBERTv2‡ [2]	—	0.397	—	—

Full 8.8M corpus. At r=64, trained and random orthogonal projections nearly tie on MRR@10 and Recall@1000 (Δ MRR@10 $= -0.0001$; Table 1). Training slightly raises Recall@100 ($+0.0087$): more relevant documents enter the top-100 candidates, but this does not translate into an MRR@10 gain. This 8.8M comparison uses a single training seed (42), so we read it as the absence of a reliable MRR@10 advantage at this budget rather than as proof of exact equality.

100k diagnostic slice. The diagnostic slice shows a different picture. Here the trained projection leads the random orthogonal baseline by about one MRR@10 point (0.8496 vs. 0.8393; Figure 1), with the sign code scored directly, before any fp32 rerank (§9).[3] The advantage is measurable in this smaller setting (per-query paired bootstrap against random R with its own scale head, $+0.0100$, 95% CI $[+0.0055, +0.0144]$); Section 8 examines what training changes.

Other settings. We also examine projection choice at r=128 on MS MARCO and compare identity truncation with random projection across BEIR domains. At r=128 neither a trained projection nor

[3]Removing the query-side scale head changes trained R by $+0.0003$ MRR@10, so the plain sign code suffices.

identity sign coding substantially outperforms a random orthogonal rotation (trained vs. random Δ MRR@10 $= -0.0027$, 95% CI $[-0.0075, +0.0019]$; Table 1), and across nine BEIR domains the differences between $\text{sign}(d_{1:64})$ and a random orthogonal projection are small and inconsistent (median $|\Delta \text{NDCG@10}| = 0.004$; Table 2). SciFact is the only corpus with a significant NDCG@10 difference ($+0.032$, Holm-adjusted $p = 0.005$). Against fp32 ColBERTv2 on the eight domains with an fp32 run, identity sign coding retains a mean 93.8% of NDCG@10 and the random projection 93.3%: sign coding costs roughly 6%, and the choice of projection changes that cost little.

Together, the full-corpus MS MARCO result and the BEIR panel show that once document tokens are reduced to sign codes, retrieval quality is largely insensitive to the choice of projection. We turn next to what sign coding changes in the representation, and what it leaves intact.

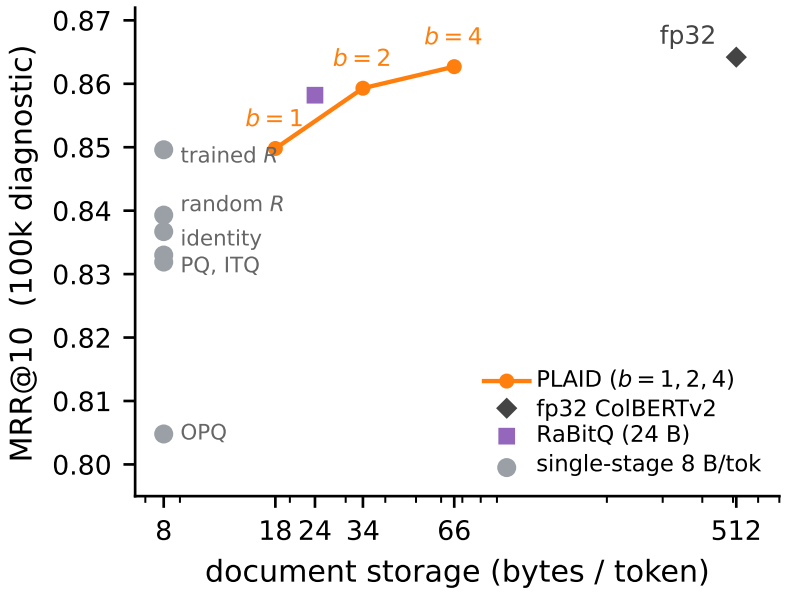


Figure 1: Single-stage storage–quality on the 100k diagnostic slice (MRR@10 vs. document bytes per token, log scale; values in Tables 1 and 7). At 8 B/tok the sign codes (trained R, random R, identity) and the matched-byte compression baselines (PQ, ITQ, OPQ) trail the larger PLAID residual codes, RaBitQ, and fp32 ColBERTv2 in quality, at substantially lower storage. Trained R is about 0.01 MRR@10 above random R.

Table 2
Per-corpus NDCG@10 at r=64, 8 B/tok on nine BEIR domains: fp32 ColBERTv2, identity-truncated sign codes ($\text{sign}(d_{1:64})$), and a random orthogonal R. $\Delta = \text{sign}(d_{1:64})-$ random (positive favours $\text{sign}(d_{1:64})$); ret. $= \text{sign}(d_{1:64})$/fp32. This comparison uses two non-learned projections, unlike the trained vs. random comparison of Table 1. Sign coding retains a mean 93.8% of fp32 NDCG@10 (random projection 93.3%), and the choice of projection has little effect; retention above 100% (webis-touche2020) may reflect small-n variance. NQ's fp32 baseline is omitted for compute ("$-$"); per-query NDCG@10 arrays are published for paired-bootstrap CIs.

Corpus	n_q	fp32	sign	rand	Δ	ret.
scifact	300	0.6464	0.6308	0.5988	**+0.032**	97.6%
trec-covid	50	0.8007	0.7841	0.7693	+0.015	97.9%
fiqa	648	0.3473	0.2999	0.3145	−0.015	86.4%
webis-touche2020	49	0.2697	0.2770	0.2659	+0.011	102.7%
quora	5000	0.8525	0.8239	0.8278	−0.004	96.6%
scidocs	1000	0.1581	0.1403	0.1420	−0.002	88.7%
nq	3452	−	0.5327	0.5312	+0.002	−
nfcorpus	323	0.3301	0.3125	0.3140	−0.001	94.7%
arguana	1406	0.3336	0.2871	0.2880	−0.001	86.0%

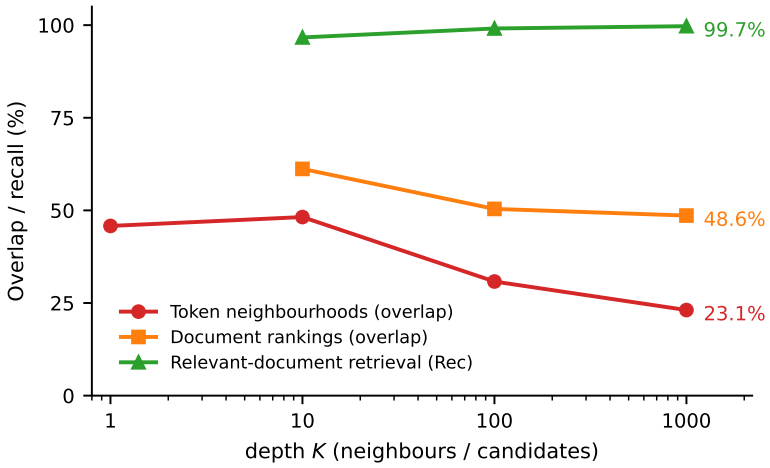


Figure 2: Token-neighbourhood and document-ranking overlap with fp32 (random orthogonal R), alongside absolute relevant-document recall (trained R), at r=64 on the 100k diagnostic slice. The overlaps measure agreement with fp32; recall measures retrieval of relevant documents and remains near the ceiling.

Table 3
Agreement between sign-coded and fp32 document rankings per query, on the 100k diagnostic MS MARCO dev slice, r=64 asymmetric scoring. Pearson is computed over the per-pair query–document scores; Spearman, Kendall, and overlap@K compare the rankings the two scorers induce (fp32 ColBERTv2 is 1.000 by construction). PCA r=64 projects onto the top 64 principal components of the document tokens, without mean-centring. Scores remain strongly correlated while the rankings agree only moderately: sign coding preserves the approximate order, not the exact one.

Projection	Pearson	ρ	τ	ovlp@10	ovlp@100	ovlp@1000
fp32 ColBERTv2	1.000	1.000	1.000	1.000	1.000	1.000
Trained R, asym	0.788	0.740	0.550	0.657	0.567	0.558
PCA r=64	0.752	0.719	0.529	0.619	0.531	0.534
Random orth. r=64	0.692	0.645	0.465	0.612	0.504	0.486
Identity r=64	0.670	0.635	0.456	0.609	0.498	0.480

6. What sign coding preserves and what it does not

The preceding results show that retrieval quality survives sign coding. One possible account is that sign coding largely preserves the underlying representation, so retrieval proceeds on nearly the same geometry as fp32. We test that account by measuring preservation at three levels: token geometry, measured as nearest-neighbour identities among document tokens; document rankings, measured as the MaxSim-induced ordering of documents for a query; and retrieval quality, measured as whether the relevant document is retrieved (Figure 2). These levels are preserved very unevenly: token geometry is heavily disrupted, document rankings substantially changed, and only retrieval quality is nearly intact. We report random-orthogonal results at the geometry and ranking levels; Table 3 reports the other projections at the ranking level. All three levels are measured on the 100k diagnostic slice.

Token geometry. A sub-bit sign code can keep the broad directional structure of the embedding space while losing the fine ordering among the many tokens that sit close together; if retrieval quality depends on preserving local token structure, substantial damage at this level should be reflected higher up. We sample 1000 anchor document tokens from the 100k diagnostic corpus and compute each anchor's top-k neighbours under fp32 and under an r=64 random-orthogonal sign code, over a pool of 200k doc tokens. The overlap is only about half at small k (48.2% at k=10) and falls to 23.1% at k=1000 (Figure 2). That is far above the 0.5% random baseline, so the code is not noise – yet more than

half of fp32's nearest-neighbour identities are already gone at $k{=}10$. Token geometry is substantially disrupted.

Document rankings. A document's ranking is not a property of any single token neighbourhood but of the aggregate MaxSim score over all its tokens, which we therefore measure separately. The disruption is milder here but still large: only roughly half of fp32's top-1000 documents survive sign coding (random orthogonal R), against under a quarter of its token neighbours (Figure 2), with moderate rank correlations (Spearman ρ 0.63–0.74). Table 3 reports the breakdown across projections: top-K overlap runs 0.61–0.66 at $K{=}10$, drops to 0.50–0.57 at $K{=}100$, and changes little thereafter. Document rankings are substantially altered; the representation is not faithfully preserved at this level either.

Retrieval quality. Even a changed ranking need not change retrieval quality. Document-ranking agreement is only a surrogate: what the candidate set must preserve is not fp32's order but the relevant document's presence, and we measure that directly (head order, which MRR@10 depends on, is restored later by reranking, §9). At $r{=}64$ asymmetric, trained R reaches Rec@1000 $= 0.9972$ against fp32's 0.9989 (Table 5) – the relevant document is nearly always retrieved, even though only 56% of fp32's exact top-1000 recurs.

The ranking damage mainly changes the *rank* of the relevant document. Among the worst 5% of queries by MRR@10 drop (349 of 6,980, mean shift -0.527), 322 (92.3%) still hold the relevant document inside the top-10 at a worse rank; only 27 lose it from the top-10 entirely.

These measurements rule out a simple preservation account. Sign coding substantially alters local token geometry and changes many document rankings, yet relevant-document retrieval remains almost unchanged.

7. Why retrieval survives

We examine how often sign coding changes MaxSim winners for query tokens (q-tokens) and the fp32 score cost of those substitutions. All measurements in this section use the 100k diagnostic slice unless stated otherwise.

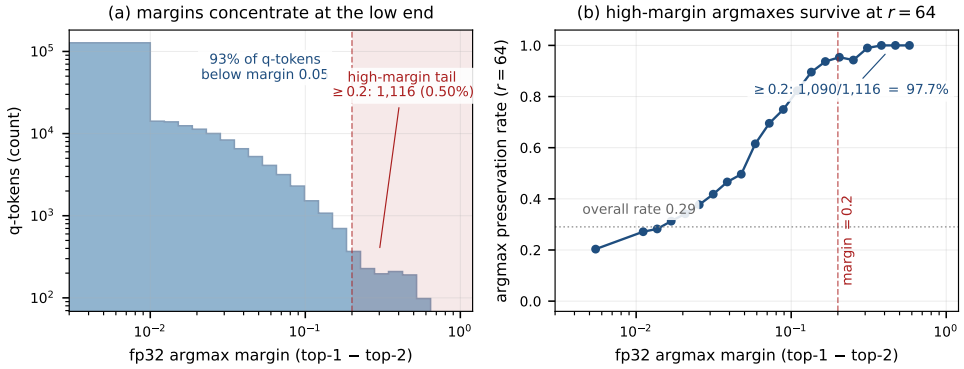


Figure 3: Argmax margin and its preservation under sign coding ($r{=}64$, 100k diagnostic slice, all 6,980 MS MARCO dev queries; tokens pooled across each query's top-100 fp32 passages). **Left:** per-query-token margins concentrate at the low end, and the high-margin tail (≥ 0.2) is a small fraction of q-tokens. **Right:** high-margin winners almost always retain their fp32 argmax under sign coding; flips occur among the low-margin near-ties, where the score cost is small.

Most winners change. MaxSim assigns each query token a winner – the document token achieving the maximum inner product – and the query–document score (Eq. 2) is the sum of those winning inner products. For the winner diagnostics in this section, each query token's winner is taken over the pooled tokens of the query's top-100 fp32 passages, rather than within one document as in Eq. 2. Because

the diagnostic pools tokens across passages, its margin distribution need not match the margins of within-document MaxSim. Sign coding replaces each document token's embedding with a sign code, so winners can change; this happens most readily for q-tokens whose fp32 winner holds only a slim lead over the runner-up. We call this fp32 lead the *margin* and mark a q-token as low-margin when it falls below 0.2. On the 6,980 dev queries against the 100k diagnostic corpus, low-margin q-tokens flip their argmax at a 71.30% rate; only 28.70% survive. The high-margin tail (margin ≥ 0.2) is only 0.50% of all q-tokens (Figure 3). With most winners changing, we next measure how much fp32 score each replacement gives up.

Most replacements are near-equal in score. A small fp32 margin guarantees one close runner-up, but not that the sign-coded substitute is that runner-up. We therefore measure where each substitute sits relative to the original winner, first in fp32 rank and then in fp32 score.

Table 4 shows that substitute winners are not concentrated among the top few fp32 runners-up. Among the 158,452 flipped low-margin q-tokens, only 18.9% of substitute winners ranked second under fp32; 38.7% ranked beyond tenth, and the median rank of the substitute is 7. The dense plateau of near-equal-score tokens produces a distribution of substitute ranks that is spread across the head, not clustered near the top. Rank is therefore the wrong measure of substitution cost: the substitute often sits several places below the winner in fp32 rank but close to it in fp32 score, and the cost is the score difference.

Table 4
Rank of the substitute winner under fp32 scoring when a flipped low-margin q-token's sign-coded argmax differs from its fp32 argmax (158,452 flipped tokens, r=64, 100k diagnostic slice; tokens pooled across each query's top-100 fp32 passages). The substitute is close to the original in fp32 *score* but widely dispersed in fp32 *rank*: rarely the runner-up, often beyond rank 10. The low-margin head is a dense plateau of near-equal tokens, not a short list of close runners-up.

Substitute fp32 rank	Share of flipped tokens
2	18.9%
3–5	25.4%
6–10	17.0%
Beyond 10	38.7%
Median rank	7

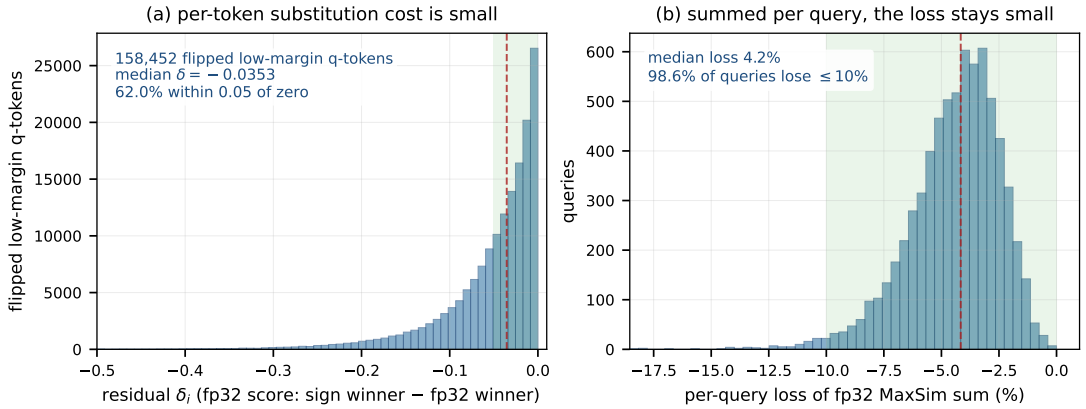


Figure 4: Direct score residual of flipped low-margin q-tokens (r=64, 100k diagnostic slice; tokens pooled across each query's top-100 fp32 passages). **Left:** for each q-token, δ_i is the fp32 score forfeited by selecting the sign-coded winner rather than the fp32 winner ($\delta_i \leq 0$ by construction), almost always a small loss. **Right:** aggregated per query, the loss is a small fraction of the pooled-token MaxSim total. The substitution cost is small per token and remains small after aggregation.

The score loss is small. For each flipped low-margin q-token, δ_i is the fp32 score of the sign-coded winner minus the fp32 score of the fp32 winner – negative by construction, and near zero when the substitution costs almost nothing in fp32 units. Among the $158{,}452$ flipped low-margin q-tokens the median δ_i is -0.0353, and 62.0% lie within 0.05 fp32-score units of zero (Figure 4, left). Summed per query, the median aggregate loss is 4.2% of the pooled-token fp32 MaxSim sum, and 98.6% of queries lose at most 10% (Figure 4, right).

The relevant quantity is therefore not argmax preservation but the size of the score residual: most winners change, yet the score they contribute changes very little. We propose this small residual as a possible link between the token-level disruption of §6 and the near-intact relevant-document recall (random R: Recall@1000 $= 0.9963$ vs. fp32 0.9989; Table 1); we report the association, not a controlled causal test.

7.1. Where the pooled-token MaxSim sum sits

If low-margin substitution explains why retrieval survives, low-margin query tokens should account for most of the pooled-token MaxSim sum. We test this by measuring their contribution relative to the high-margin tail.

On the $6{,}980$ dev queries against the 100k diagnostic slice at r=64, high-margin q-tokens are 0.50% of the pooled population (median query carries zero) and contribute 0.51% of the pooled-token MaxSim sum (median 0.0%). The sum sits on the low-margin bulk.

The high-margin tail is also where preservation is strongest: those q-tokens keep 97.67% of their fp32 argmaxes and 98.05% of their sum contribution under sign coding. But that stability acts on only a 0.51% slice. The low-margin bulk that carries the rest flips at 71.30%, and its per-token residuals stay near zero (62.0% within 0.05 fp32-score units; Figure 4).

Why projection choice barely matters. The small measured substitution costs for random R suggest limited room for a learned projection to reduce them. We report the low-margin residuals for the random projection on the 100k slice. The near-ties across all three projections at 8.8M (§5) are consistent with the same low-margin regime holding for the other projections and at full scale, though we do not report the decomposition there.

Second-encoder check. A second encoder provides a limited direction check on the mechanism. ConstBERT$_{32}$ [17], a ColBERT-style multi-vector model with a fixed set of C=32 document vectors, has a somewhat thinner high-margin tail than ColBERTv2 in our 100k diagnostic probe: its fraction of per-q-token argmaxes at margin ≥ 0.2 is 0.35%, compared with ColBERTv2's 0.50%. Applying a random orthogonal R built the same way (seed 42) at r=64, ConstBERT retains 93.2% of its fp32 MRR@10 ($0.858 \rightarrow 0.800$). At lower r the reported gap between the two encoders widens: ColBERTv2 retains 0.773 at r=32 and 0.453 at r=16 (89.5% and 52.4% of its fp32 reference), while ConstBERT falls to 0.628 (73.2%) and 0.222 (25.9%). This is consistent with robustness depending on how much of the score sits at low margin, but with only two encoders it is a consistency check, not a predictive test. The 8.8M experiment was not re-run on ConstBERT.

8. What training changes

If low-margin substitution explains why retrieval survives, and the choice of projection barely affects retrieval (§5), what does training change? On the 100k slice trained R does lead the random orthogonal baseline – MRR@10 0.8496 vs. 0.8393; per-query paired bootstrap against random R with its own scale head gives $+0.0100$ (95% CI $[+0.0055, +0.0144]$, $p < 0.001$, n=6,980). We examine how training changes the scorer (Figure 5).

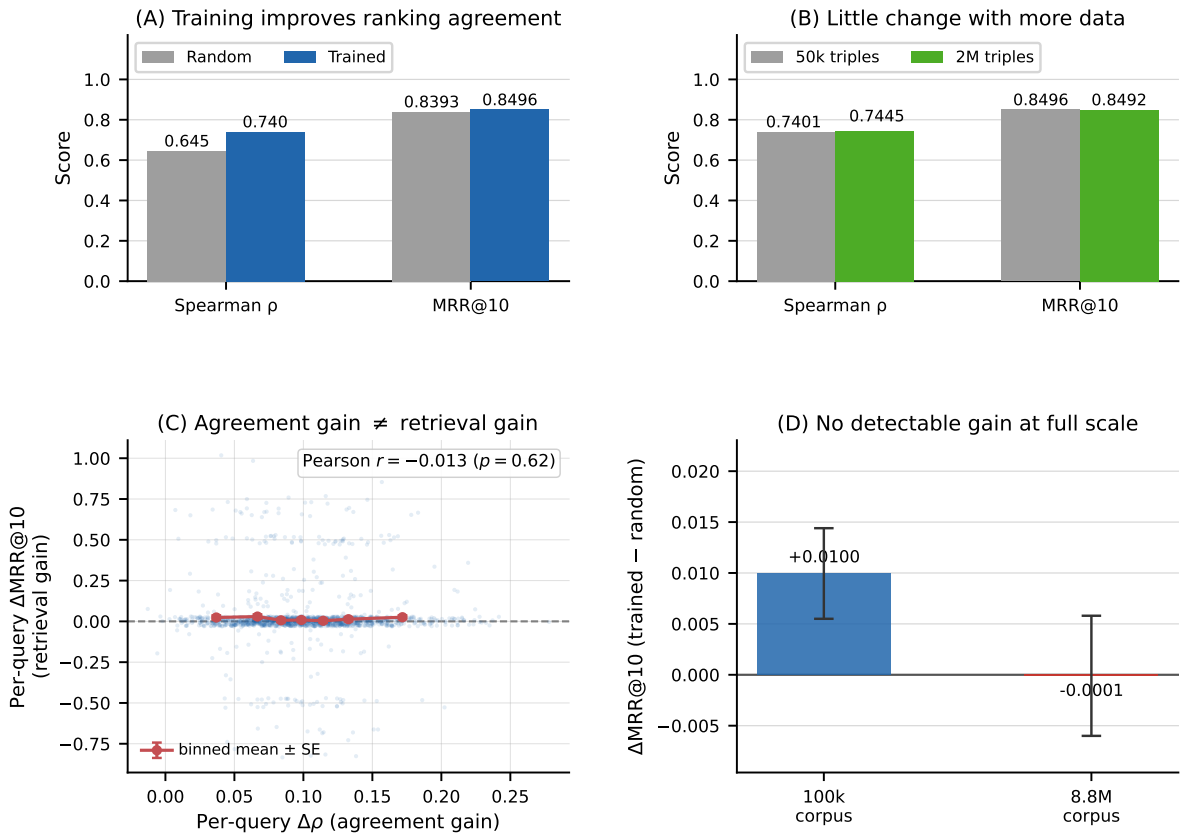


Figure 5: What training changes, and what it does not (r=64; panels A–C use the 100k diagnostic slice). **(A)** A trained R raises Spearman ρ with fp32 rankings and MRR@10 over a random orthogonal R. **(B)** Ranking agreement changes little with $40\times$ more training data (50k vs. 2M triples). **(C)** Across a 1,500-query sample the per-query gain in ranking agreement, $\Delta\rho$, is essentially uncorrelated with the per-query retrieval gain ΔMRR@10 (Pearson $r = -0.013$, $p = 0.62$; points jittered vertically, line is the binned mean $\pm$ SE). **(D)** The trained-vs-random advantage is $+0.0100$ on the 100k slice but -0.0001 at the full 8.8M corpus (95% CIs).

Training improves agreement with fp32 rankings. Relative to a random orthogonal projection, the trained projection raises Spearman ρ from 0.64 to 0.74, Kendall τ from 0.47 to 0.55, and overlap@10 from 0.61 to 0.66 (Table 3). The trained projection differs from the PCA basis by approximately 21°.

The improvement changes little with more data. A checkpoint trained on 2M triples – $40\times$ the data – reaches essentially the same ranking agreement (ρ 0.7445 vs. 0.7401) and the same 100k MRR@10 (0.8492 vs. 0.8496).

Agreement gains do not predict which queries improve. Per query, the gain in ranking agreement is essentially unassociated with the change in MRR@10 (Pearson $r = -0.013$, $p = 0.62$): queries where trained R gains MRR@10 and queries where it loses show the same mean agreement gain ($+0.100$ vs. $+0.096$). Per-query changes in MaxSim score loss, argmax-flip rate, and substitute rank (§7) are likewise uncorrelated with changes in MRR@10 ($|r| < 0.04$). Nor does training make relevant documents' codes more confident: relevant and non-relevant document tokens sit the same mean distance from the trained hyperplanes (0.08017 vs. 0.08018).

No detectable trained-vs-random gain at full scale. The $+0.0100$ on the 100k slice is -0.0001 at full 8.8M (95% CI $[-0.0060, +0.0058]$). Training consistently improves agreement between sign-coded and fp32 rankings on the 100k slice, but we detect no MRR@10 gain at full scale. The 100k slice is also

the training passage pool, and the checkpoint was selected on its dev queries (§4), so part of the 100k gain may reflect fitting to that slice.

This disconnect is not specific to training: among the matched-byte baselines (Table 1), OPQ is optimised for reconstruction error yet scores below plain PQ (0.8048 vs. 0.8330), another case where faithful reconstruction does not track retrieval quality. The pattern is consistent with training improving ordering within an already-retrievable candidate set.

9. A projection-training-free retrieval pipeline

Because sign coding largely preserves the candidate set but changes its ordering (§6–§7), we evaluate a two-stage pipeline with sign-coded candidate generation and full-dimensional reranking.

On the 100k diagnostic slice, sign-coded MaxSim at r=64 (8 B/tok; throughout this section "sign-coded" denotes this r=64 code) retrieves a top-K at low byte budget; fp32 MaxSim then rescores those K at full precision. Stage 2 computes fp32 MaxSim only for the K candidates of each query, cheap when $K \ll N$.

Table 5
Two-stage retrieval on the 100k diagnostic MS MARCO dev slice (r=64). Stage 1 is sign-coded asymmetric MaxSim retrieving the top-1000 at 8 B/tok; stage 2 rescores the top-K at fp32. Reranking 100 candidates recovers fp32-level MRR@10. Rec@1000 here is the two-stage ceiling set by the stage-1 top-1000, not exhaustive fp32 retrieval over the slice (Table 1).

Pipeline	MRR@10	Rec@100	Rec@1000
fp32 MaxSim only	0.8641	0.9940	0.9989
Sign-coded, single stage (r=64)	0.8496	0.9910	0.9972
Sign-coded → fp32 rerank K=100	**0.8642**	**0.9910**	**0.9972**

The rerank closes the gap. Stage 1 alone reaches MRR@10 = 0.8496, 0.0145 below fp32's 0.8641. An fp32 rerank of the sign-coded top-100 closes the gap to within 0.0001 (0.8642 vs. 0.8641). Rec@1000 is 0.9972 after stage 1 and unchanged after rerank at K=100; Rec@100 also remains at 0.9910 because the rerank reorders the top 100 candidates without changing which documents they contain. Sign-coded Rec@100 (0.9910) and Rec@1000 (0.9972) fall slightly short of the full fp32 values (0.9940 and 0.9989), so the candidate set is not identical to fp32 retrieval. The stage-1 projection does not matter to the MRR@10 result: a random orthogonal R reaches MRR@10 = 0.8641 after the same K=100 rerank, against 0.8642 for trained R (the table uses trained R for stage 1; a paired bootstrap at corpus scale follows below).

It holds at full scale. Single-stage MRR@10 is 0.3723 for trained R and 0.3724 for random orthogonal R at 8 B/tok. A full-dimensional rerank of the top-100 sign-coded candidates[4] lifts random R to 0.4002 and trained R to 0.3997, both reaching the published ColBERTv2 level reported by [2] (≈0.397) from the 8 B/tok index. Trained and random R show no statistically reliable MRR@10 difference after rerank: paired bootstrap Δ MRR@10 = -0.0005 (95% CI $[-0.0012, +0.0002]$, n=6,980) at K=100, and $\Delta = +0.0000$ (95% CI $[-0.0005, +0.0006]$) at K=1000 where both reach 0.4001; both intervals include zero. For this trained checkpoint, projection training does not improve MRR@10 beyond reranking at corpus scale.

Where the residual misses concentrate. Stage 2 orders the same candidates at full precision, recovering the head that low-margin flips had shuffled. On the 100k slice, the 169 queries (of 6,980) that still miss after K=100 are markedly more near-tied at the document level: their median fp32 top-1

[4] At 8.8M, the rerank reads the stored full-dimensional fp32 embeddings and computes MaxSim in fp16 on the GPU.

vs. top-2 MaxSim margin is 0.56, compared with 3.68 for resolved queries, roughly 6.6× smaller. These are the query-level analogues of the low-margin q-tokens of §7.1 – the fp32 rerank has limited room to separate them when their fp32 scores are near-tied.

Storage. The 8 B/tok is the stage-1 candidate index, scored for every query against every document token and so RAM-resident – the tier sign coding shrinks, and the same tier PLAID holds at 34 B/tok (b=2). The fp32 vectors stage 2 needs (≈ 512 B/tok) are read only for the K≈100 reranked candidates per query, so they can sit on a cold SSD tier or be re-encoded on demand. Total document-side storage is therefore not reduced: the saving is in the always-resident candidate index, the binarise-then-rescore tiering that binary-quantised vector databases already use [5]. BPR also uses a two-stage search-and-rerank design, but reranks binary passage codes against a continuous query embedding rather than storing fp32 passage vectors for this purpose [22].

Stage-2 rerank latency. The rerank scores only the top K candidates, so its cost scales with K, not the corpus. Under the same interleaved float harness as the full scan (Table 8; 32-token augmented queries, median of 5 round medians), the stage-2 fp32 rerank costs 1.1 ms at K=100, 2.1 ms at K=256, and 8.5 ms at K=1024 (Table 6); at K=100 that is roughly 130× below the 144 ms exhaustive fp32 scan (Table 8). Candidate generation is the separate stage-1 cost, so this is the rerank-stage figure alone.

Table 6
Stage-2 fp32 rerank latency over the top-K candidates (100k diagnostic slice; 32-token augmented queries, Apple M4 Pro, 48 GB RAM, macOS 26.0.1, PyTorch 2.8.0, 8-thread CPU; median of 5 interleaved round medians (each the median over 500 queries), IQR across rounds in parentheses; same harness as Table 8). Latency scales with K, not corpus size; at K=100 it is two orders of magnitude below an exhaustive fp32 scan (Table 8).

Configuration	median ms (IQR)
Stage-2 fp32 rerank, $K = 100$	1.1 (0.25)
Stage-2 fp32 rerank, $K = 256$	2.1 (0.29)
Stage-2 fp32 rerank, $K = 1024$	8.5 (0.25)

9.1. Comparison with PLAID

If training does not improve full-scale MRR@10 over a random projection and a full-dimensional rerank closes the head ordering gap, the practical question is whether the sign code offers a useful storage–quality operating point. Sign coding is a per-token storage format, not a retrieval engine: it sets how cheaply each document token is stored and scored, not which candidates are selected – candidate pruning (PLAID's centroid pruning [6]) is a separate candidate-generation stage that we do not replace. We isolate this compression layer by scoring all codecs exhaustively over the 100k diagnostic slice. Table 7 compares the sign code with our reimplementation of ColBERTv2's b-bit residual codes on storage and quality, following the efficiency–effectiveness comparisons of MacAvaney and Tonellotto [45].

A random orthogonal R scores MRR@10 = 0.8393 single-stage, below PLAID b=2; the fp32 rerank of its top-100 lifts it to fp32 parity, 0.8641 (§9), from an 8 B/tok index – 4.25× smaller than PLAID b=2's 34 B/tok and with no centroid table.[5] A comparable rerank may also lift the residual codec (we do not evaluate that variant), so the saving is not in quality but in the candidate-index storage that reaches it. Trained R without the scale head reaches 0.8500 single-stage, but the rerank closes the difference (0.8642 vs. 0.8641 at K=100).

Storage accounting assumptions. The 4.25× win is a document-side, candidate-index figure; the per-token bytes exclude PLAID's centroid codebook, which is amortised across the corpus and so

[5]The sign code's own amortised R matrix at r=64 ($64 \times 128 \times 4 = 32$ KiB) is below 5×10^{-3} B/tok at 100k and negligible.

Table 7
Per-token storage–quality vs. PLAID at 100k MS MARCO dev (ColBERTv2 encoder). Our recipe (bold) is a random orthogonal R at 8 B/tok with an fp32 top-100 rerank: it reaches fp32-parity MRR@10 from a $4.25\times$ smaller candidate index than PLAID b=2, with no projection training. Single-stage, random R sits below PLAID b=2 and trained R just above PLAID b=1, but the rerank closes the gap, so the trained row is reported only for reference. (The recipe's random R carries no scale head, unlike the trained-vs-random comparison at 8.8M.) PLAID b=1 is ColBERTv2's 1-bit residual-quantisation mode [2], the established sub-byte multi-vector compressor; the identity-sign row is the no-rotation, raw sign-coding baseline used by deployed multi-vector databases. The PLAID $b \in \{1, 2, 4\}$ rows are our codec re-implementation (centroid+b-bit residual, C=32,768) on the shared frozen embeddings, exhaustively scored – the codec, not the engine (Table 1); the Pareto framing follows [45] but the numbers are not transcribed from it. Centroid codebooks are amortised across the corpus and excluded from per-token bytes; the amortisation is non-negligible at 100k (PLAID b=2 all-in ≈ 36.5 B/tok; see the storage accounting paragraph).

Method	B/tok	MRR@10	NDCG@10
fp32 ColBERTv2	512	0.8641	0.8889
PLAID b=4	66	0.8627	0.8875
PLAID b=2	34	0.8593	0.8841
PLAID b=1	18	0.8498	0.8756
Random orthogonal R, r=64	8	0.8393	0.8670
+ fp32 rerank (K=100)	8	**0.8641**	–
Trained R, r=64 (this work, no scale)	8	0.8500	0.8755
Identity sign, r=64	8	0.8367	0.8644

shrinks with corpus size.[6] As §9 notes, this is the always-resident candidate index; the stage-2 fp32 store is a separate cold tier.

Retrieval latency. Scoring a residual codec first reconstructs an approximate fp32 token – a centroid plus a dequantised residual – and then scores it at the full 128 dimensions; the sign code is scored directly as a 64-dimensional vector, half the width and with no reconstruction. Scored exhaustively, the sign code takes 119.5 ms/query against 144.2 ms for fp32 ColBERTv2, a $1.21\times$ speedup (Table 8). On quality the recipe, the official PLAID engine (0.8631), and uncompressed fp32 (0.8641) are within 0.0011 MRR@10 of one another. The engine itself runs far below any exhaustive scan (8.0 ms median, about $15\times$ faster than the exhaustive sign-coded scan) because it prunes to at most 4,096 candidates first – a step that could be combined with the sign code, though we do not evaluate that integration. Storing document tokens as sign bits and scoring them against full-precision queries is already a deployed pattern: Vespa and Elasticsearch both offer binarised document-side token vectors scored with asymmetric or Hamming late interaction [3, 33, 4]. Using the 8 B/tok sign code inside such a pruned engine, and keeping the smaller index, is the open step.

10. Discussion

Where encoder sensitivity would come from. The pooled-token MaxSim sum sits on the low-margin bulk, not the high-margin tail (§7.1). An encoder's tolerance to small r should therefore track how much of its score sits at very low margin, where flips are frequent and grow costlier as the code coarsens; the high-margin fraction is only an indirect summary of this. Across our two encoders that summary moves in the expected direction: ConstBERT$_{32}$, with a thinner high-margin tail (0.35% vs. ColBERTv2's 0.50% of q-tokens at margin ≥ 0.2), degrades faster as r shrinks (§7). But the statistic separates the two encoders by little, so we read it as consistent with the account rather than a predictor.

[6] The codebook is C=32,768 centroids $\times$ 128 dims $\times$ 4 bytes = 16 MiB. At the 100k diagnostic ($\approx$ 6.73M tokens) it adds $\approx$ 2.5 B/tok, raising PLAID b=2 from 34 to $\approx$ 36.5 B/tok and the ratio to $\approx 4.6\times$; at 8.8M ($\approx$ 597M tokens) it amortises to $\approx$ 0.03 B/tok and the all-in ratio approaches the bare $4.25\times$.

Table 8
Per-query retrieval latency on the 100k diagnostic MS MARCO dev slice (32-token augmented queries on an Apple M4 Pro, 48 GB RAM, macOS 26.0.1, PyTorch 2.8.0, 8-thread, batch 1, CPU; median of 5 interleaved round medians (each the median over 500 queries), IQR across rounds in parentheses; query encoding excluded). The two exhaustive rows isolate the distance kernel (64-dimensional sign code vs. 128-dimensional fp32); residual-codec rows are omitted because, scored exhaustively, each reconstructs an fp32 token for full-dimensional scoring. [§]The official PLAID engine (colbert-ai 0.2.22, b=2, same machine) prunes to at most 4,096 candidates, so it does different work from the exhaustive rows; on the same passages its quality is within 0.0011 MRR@10 of fp32.

Configuration	B/tok	median ms (IQR)
Sign-coded r=64, exhaustive	8	**119.5** (0.5)
fp32 ColBERTv2, exhaustive	512	144.2 (0.3)
PLAID engine, official (b=2, pruned)[§]	≈ 34	8.0 (0.4)

We have not tested newer encoders such as ModernColBERT and LateOn. LightOn reports poor performance for MUVERA and SMVE on these models, whose token embeddings concentrate in a narrow cone; mean-centring helps but does not resolve the problem [46]. Random hyperplanes rarely split such a cone, so our random-R sign codes may also be sensitive to this geometry. The ConstBERT$_{32}$ check does not test this geometry. Testing on these encoders, and measuring their effective rank and substitution costs alongside retrieval quality, is future work.

What the decomposition suggests for stage-1 compressors. In our decomposition, the high-margin tail largely survives sign coding but carries only 0.51% of the pooled-token sum (§7.1); the low-margin bulk carries the other $\approx 99.5\%$. This suggests that, for this encoder and code, a stage-1 compressor may not need to preserve every fp32 token winner. It may be enough to keep substitutions to a small local change in score across that bulk, a weaker target than accurate per-token reconstruction. Designing compressors that target this bulk directly is a possible direction.

What the trained-vs-random result does and does not show. The observed convergence of trained and random projections holds for the sign-coded settings studied here. On the 100k slice a trained projection does win (§8), but the per-query gain in ranking agreement is uncorrelated with the change in MRR@10 (Pearson $r = -0.013$), and the advantage is not visible at 8.8M. We do not read this as evidence that trained projections are useless in general: under higher-bit residual codes, which preserve more of the fp32 score, learned projection blocks still report gains [31]. The full-scale result resembles other negative results in retrieval, such as the finding that scaling pointwise cross-encoder reranking does not monotonically improve effectiveness [47].

Scope and limitations. The mechanism comes from one encoder family (ColBERTv2 on MS MARCO), with ConstBERT$_{32}$ as a single cross-check at 100k scale but not at the full 8.8M. The 8.8M trained-vs-random comparison rests on one training seed (42), trained on random rather than mined hard negatives, so the learned checkpoint is a budgeted one rather than an exhaustively tuned one. The small measured substitution costs for random R (§7) suggest these choices may matter little, but we have not tested alternative negative-mining schemes or other encoder families. Whether the account generalises beyond the ColBERT family remains open. The BEIR panel (Table 2) is a non-learned projection-choice comparison, not a trained-vs-random comparison, and covers nine of thirteen corpora; the four largest (HotpotQA, DBPedia-Entity, FEVER, Climate-FEVER) were excluded for compute. SciFact has the largest NDCG@10 difference (+0.032); the other eight fall within $|\Delta\,\text{NDCG@10}| \leq 0.015$.

Our sign-code and fp32 latency measurements are single-CPU, batch-1, and exhaustive (§9, §9.1); none extends to GPU, distributed, or pruned serving. In our local run, the official PLAID engine with pruning is much faster than any exhaustive scan. Sign coding is a per-token storage format, complementary to such engines rather than a replacement, and combining it with candidate pruning at full scale remains

future work. We hold *asymmetric* scoring (fp32 queries, sign-coded documents) fixed throughout; the fully binary popcount variant is a separate speed–quality point, costing 0.0132 MRR@10 single-stage ($0.8496 \to 0.8364$) at $r{=}64$. The 8.8M ColBERTv2 reference (0.397) is the level reported by [2], not an in-harness measurement.

11. Conclusion

Sign coding substantially disrupts token geometry while largely preserving retrieval quality. This is consistent with low-margin substitution: most winners in the pooled-token diagnostic change, but replacements usually have a nearby fp32 score, so their summed fp32 score barely moves. Many prior compressors are built to track the fp32 reference per token; at sub-bit sign coding the diagnostic sum survives even when token neighbourhoods, argmaxes, and document rankings are substantially altered. These results suggest that exact token-level similarity preservation is not necessary for effective sign-coded retrieval.

Three comparisons show limited effects of projection choice on retrieval. At 8.8M, trained and random projections show no statistically reliable difference (Δ MRR@10 $= -0.0001$, 95% CI $[-0.0060, +0.0058]$). At $r{=}128$, identity sign coding and a random rotation differ by only 0.0005 MRR@10 (0.3921 vs. 0.3916). Across nine BEIR domains identity-truncated sign codes and a random projection differ by a median $|\Delta\,\text{NDCG@10}| = 0.004$. Training improves agreement with fp32 rankings on the 100k slice (§8), but yields no detectable MRR@10 gain at full scale.

The practical result is a projection-training-free recipe: a random orthogonal R at 8 B/tok generates candidates, and a full-dimensional rerank of the top 100 recovers fp32-level MRR@10 at 100k and reaches the published ColBERTv2 reference level at 8.8M, with a candidate index $4.25\times$ smaller than PLAID's 2-bit residual codec. The recipe is complementary to pruning-based engines such as PLAID; combining the smaller index with candidate pruning is the open engineering task.

Two questions remain. Multi-bit residual coding may restore a role for training: more bits lower the argmax-flip rate, so fewer winners change and more of the remaining error lies in the ordering of top-ranked candidates, which training can improve. And encoder coverage is thin: the link between an encoder's margin distribution and its robustness rests on two ColBERT-family encoders, and its generalisation to other embedding geometries remains untested.

Declaration on Generative AI

During the preparation of this work, the authors used Claude (Anthropic) and Codex (OpenAI) in order to: Paraphrase and reword, Improve writing style, Grammar and spelling check, Citation management, and Formatting assistance. The authors also used Claude to help write the code for the experiments. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

References

[1] O. Khattab, M. Zaharia, ColBERT: Efficient and effective passage search via contextualized late interaction over BERT, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM, 2020, pp. 39–48. doi:10.1145/3397271.3401075.

[2] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, M. Zaharia, ColBERTv2: Effective and efficient retrieval via lightweight late interaction, in: Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), ACL, 2022, pp. 3715–3734. doi:10.18653/v1/2022.naacl-main.272.

[3] J. K. Bergum, Announcing the Vespa ColBERT embedder, Vespa Blog, 2024. URL: https://blog.vespa.ai/announcing-colbert-embedder-in-vespa/.

[4] P. Straßer, B. Trent, Scaling late interaction models in Elasticsearch – part 2, Elastic Search Labs Blog, 2025. URL: https://www.elastic.co/search-labs/blog/late-interaction-model-colpali-scale.

[5] S. Aquino, Advanced retrieval with ColPali & Qdrant vector database, Qdrant Blog, 2024. URL: https://qdrant.tech/blog/qdrant-colpali/.

[6] K. Santhanam, O. Khattab, C. Potts, M. Zaharia, PLAID: An efficient engine for late interaction retrieval, in: Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM), ACM, 2022, pp. 1747–1756. doi:10.1145/3511808.3557325.

[7] F. M. Nardini, C. Rulli, R. Venturini, Efficient multi-vector dense retrieval with bit vectors, in: Advances in Information Retrieval: 46th European Conference on Information Retrieval (ECIR), Springer, 2024, pp. 3–17. doi:10.1007/978-3-031-56060-6_1. arXiv:2404.02805.

[8] Y. Gong, S. Lazebnik, Iterative quantization: A procrustean approach to learning binary codes, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2011, pp. 817–824. doi:10.1109/CVPR.2011.5995432.

[9] J. Gao, C. Long, RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, Proceedings of the ACM on Management of Data (SIGMOD) 2 (2024) 1–27. doi:10.1145/3654970.

[10] J. L. Scheerer, M. Zaharia, C. Potts, G. Alonso, O. Khattab, WARP: An efficient engine for multi-vector retrieval, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM, 2025, pp. 2504–2512. doi:10.1145/3726302.3729904. arXiv:2501.17788.

[11] S. Martinico, F. M. Nardini, C. Rulli, R. Venturini, Efficient multivector retrieval with token-aware clustering and hierarchical indexing, in: Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2026, pp. 3982–3987. doi:10.1145/3805712.3809927. arXiv:2604.28142.

[12] C. Lassance, M. Maachou, J. Park, S. Clinchant, A study on token pruning for ColBERT, arXiv preprint arXiv:2112.06540 (2021). arXiv:2112.06540.

[13] Y. Zong, B. Piwowarski, Towards lossless token pruning in late-interaction retrieval models, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2025, pp. 2407–2417. doi:10.1145/3726302.3730100. arXiv:2504.12778.

[14] A. Acquavia, C. Macdonald, N. Tonellotto, Static pruning for multi-representation dense retrieval, in: Proceedings of the ACM Symposium on Document Engineering (DocEng), 2023, pp. 1–10. doi:10.1145/3573128.3604896.

[15] Y. Kankanampati, Y. Zong, N. Tomeh, B. Piwowarski, J. Le Roux, A Voronoi cell formulation for principled token pruning in late-interaction retrieval models, in: Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2026, pp. 756–766. doi:10.1145/3805712.3809726. arXiv:2603.09933.

[16] S. Hofstätter, O. Khattab, S. Althammer, M. Sertkan, A. Hanbury, Introducing neural bag of whole-words with ColBERTer: Contextualized late interactions using enhanced reduction, in: Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM), 2022, pp. 737–747. doi:10.1145/3511808.3557367. arXiv:2203.13088.

[17] S. MacAvaney, A. Mallia, N. Tonellotto, Efficient constant-space multi-vector retrieval, in: Proceedings of the 47th European Conference on Information Retrieval (ECIR), 2025, pp. 237–245. doi:10.1007/978-3-031-88714-7_22. arXiv:2504.01818.

[18] H. Qin, A. Martin, R. Jha, C. Zuo, R. Kriz, B. Van Durme, Multi-vector index compression in any modality, in: Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2026, pp. 1519–1530. doi:10.1145/3805712.3809589. arXiv:2602.21202.

[19] J. Veneroso, R. Jayaram, J. Rao, G. Hernández Ábrego, M. Hadian, D. Cer, CRISP: Clustering multi-vector representations for denoising and pruning, arXiv preprint arXiv:2505.11471 (2025). arXiv:2505.11471.

[20] B. Clavié, A. Chaffin, G. Adams, Reducing the footprint of multi-vector retrieval with minimal performance impact via token pooling, arXiv preprint arXiv:2409.14683 (2024). arXiv:2409.14683.

[21] L. Dhulipala, M. Hadian, R. Jayaram, J. Lee, V. Mirrokni, MUVERA: Multi-vector retrieval via fixed dimensional encodings, in: Advances in Neural Information Processing Systems (NeurIPS), 2024, pp. 101042–101073. doi:10.52202/079017-3204. arXiv:2405.19504.

[22] I. Yamada, A. Asai, H. Hajishirzi, Efficient passage retrieval with hashing for open-domain question answering, in: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers), Association for Computational Linguistics, 2021, pp. 979–986. doi:10.18653/v1/2021.acl-short.123.

[23] H. Jégou, M. Douze, C. Schmid, Product quantization for nearest neighbor search, IEEE Transactions on Pattern Analysis and Machine Intelligence 33 (2011) 117–128. doi:10.1109/TPAMI.2010.57.

[24] T. Ge, K. He, Q. Ke, J. Sun, Optimized product quantization for approximate nearest neighbor search, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2013, pp. 2946–2953. doi:10.1109/CVPR.2013.379.

[25] J. Zhan, J. Mao, Y. Liu, J. Guo, M. Zhang, S. Ma, Jointly optimizing query encoder and product quantization to improve retrieval performance, in: Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM), 2021, pp. 2487–2496. doi:10.1145/3459637.3482358. arXiv:2108.00644.

[26] M. S. Charikar, Similarity estimation techniques from rounding algorithms, in: Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing (STOC), 2002, pp. 380–388. doi:10.1145/509907.509965.

[27] W. B. Johnson, J. Lindenstrauss, Extensions of Lipschitz mappings into a Hilbert space, Contemporary Mathematics 26 (1984) 189–206. doi:10.1090/conm/026/737400.

[28] D. Achlioptas, Database-friendly random projections: Johnson–Lindenstrauss with binary coins, Journal of Computer and System Sciences 66 (2003) 671–687. doi:10.1016/S0022-0000(03)00025-4.

[29] A. Andoni, P. Indyk, Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions, Communications of the ACM 51 (2008) 117–122. doi:10.1145/1327452.1327494.

[30] L. Caspari, M. Dinzinger, K. Ghosh Dastidar, C. Fellicious, J. Mitrović, M. Granitzer, CoRECT: A framework for evaluating embedding compression techniques at scale, in: Advances in Information Retrieval: 48th European Conference on Information Retrieval (ECIR), Springer, 2026, pp. 383–398. doi:10.1007/978-3-032-21321-1_48. arXiv:2510.19340.

[31] B. Clavié, S. Lee, R. Takehi, A. Shakir, M. P. Kato, Simple projection variants improve ColBERT performance, arXiv preprint arXiv:2510.12327 (2025). arXiv:2510.12327.

[32] Y. Chen, Y. Zhang, H. Guo, R. Tang, I. King, An effective post-training embedding binarization approach for fast online top-K passage matching, in: Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 2: Short Papers), Association for Computational Linguistics, 2022, pp. 102–108. doi:10.18653/v1/2022.aacl-short.14.

[33] J. K. Bergum, Scaling ColPali to billions of PDFs with Vespa, Vespa Blog, 2024. URL: https://blog.vespa.ai/scaling-colpali-to-billions/.

[34] R. Nogueira, K. Cho, Passage re-ranking with BERT, arXiv preprint arXiv:1901.04085 (2019). arXiv:1901.04085.

[35] R. Nogueira, Z. Jiang, R. Pradeep, J. Lin, Document ranking with a pretrained sequence-to-sequence model, in: Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 708–718. doi:10.18653/v1/2020.findings-emnlp.63. arXiv:2003.06713.

[36] T. Formal, S. Clinchant, H. Déjean, C. Lassance, SPLATE: Sparse late interaction retrieval, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2024, pp. 2635–2640. doi:10.1145/3626772.3657968. arXiv:2404.13950.

[37] J. Lee, Z. Dai, S. M. K. Duddu, T. Lei, I. Naim, M.-W. Chang, V. Zhao, Rethinking the role of token retrieval in multi-vector retrieval, in: Advances in Neural Information Processing Systems (NeurIPS), 2023, pp. 15384–15405. doi:10.52202/075280-0677. arXiv:2304.01982.

[38] Q. Liu, G. Guo, J. Mao, Z. Dou, J.-R. Wen, H. Jiang, X. Zhang, Z. Cao, An analysis on matching mechanisms and token pruning for late-interaction models, ACM Transactions on Information Systems 42 (2024) 1–28. doi:10.1145/3639818. arXiv:2403.13291.

[39] A. S, A. Garg, K. Shiragur, N. Kayal, Incorporating token importance in multi-vector retrieval, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2026, pp. 32860–32866. doi:10.1609/aaai.v40i39.40566. arXiv:2511.16106.

[40] A. Edy, M. Conti, Q. Macé, Working notes on late interaction dynamics: Analyzing targeted behaviors of late interaction models, arXiv preprint arXiv:2603.26259 (2026). arXiv:2603.26259.

[41] R. K. Salla, M. Saravanan, R. M. Amancherla, CrossQ: Task-aligned cross-token conditional quantization for late interaction retrieval, in: Proceedings of the 43rd International Conference on Machine Learning, volume 306 of *Proceedings of Machine Learning Research*, PMLR, 2026, pp. 107343–107377. URL: https://proceedings.mlr.press/v306/salla26a.html.

[42] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, L. Deng, MS MARCO: A human generated MAchine reading COmprehension dataset, in: Proceedings of the Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches 2016 co-located with NIPS 2016, volume 1773 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2016. URL: https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf.

[43] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, I. Gurevych, BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models, in: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, volume 1, 2021. arXiv:2104.08663.

[44] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, H. Jégou, The Faiss library, IEEE Transactions on Big Data 12 (2026) 346–361. doi:10.1109/TBDATA.2025.3618474.

[45] S. MacAvaney, N. Tonellotto, A reproducibility study of PLAID, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM, 2024, pp. 1411–1419. doi:10.1145/3626772.3657856. arXiv:2404.14989.

[46] A. Chaffin, Party is over: regularizing ColBERT models to fix efficient ANN methods, LightOn technical blog, Hugging Face, 2026. URL: https://huggingface.co/blog/lightonai/lateon-regularization.

[47] M. Jacob, E. Lindgren, M. Zaharia, M. Carbin, O. Khattab, A. Drozdov, Drowning in documents: Consequences of scaling reranker inference, 2025. arXiv:2411.11767v2, presented at the ReNeuIR Workshop at SIGIR 2025.